

Servidor TCP – Thread Pool

Nombre Estudiante: Justin David Galvis García

Código: 2225617

Se realizo el servidor tcp de dos formas una sencilla donde el usuario pone la cadena de caracteres y una más automatizada utilizando Jmeter y un servidor multi hilo para poder realizar todas las E/S del programa y guarda la información necesaria en un archivo.txt.

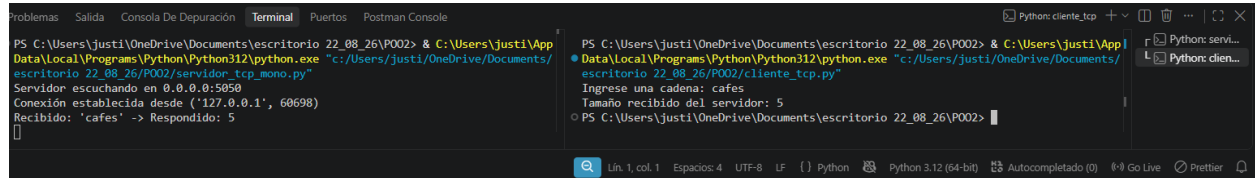
Donde el servidor debe realizar las siguientes indicaciones:

- El servidor debe aceptar mensajes TCP en un puerto determinado 5050
- El mensaje solamente es una cadena de texto que solo incluya los caracteres del alfabeto
- El servidor contara cuantas veces aparece el ultimo carácter en la cadena de texto
- Para probar el servidor usaremos JMeter o Locust
- La aplicación debe tener persistencia, guardar en un archivo de texto cada palabra recibida y el número retornado.
- Importante Investigador Cola de mensajes de espera y Pool de Threads

Primero vamos a mirar el servidor mono

```
1 import socket
2 from datetime import datetime
3
4 HOST = "0.0.0.0"
5 PORT = 5050
6 TXT_PATH = "registro_conexiones.txt"
7
8 def guardar_en_txt(addr, cadena, longitud):
9     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
10    linea = f"{timestamp},{addr[0]},{addr[1]},{cadena},{longitud}\n"
11    with open(TXT_PATH, "a", encoding="utf-8") as f:
12        f.write(linea)
13
14 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
15     s.bind((HOST, PORT))
16     s.listen()
17     print(f"Servidor escuchando en {HOST}:{PORT}")
18     try:
19         while True:
20             conn, addr = s.accept()
21             with conn:
22                 print(f"Conexión establecida desde {addr}")
23                 data = conn.recv(1024)
24                 if not data:
25                     continue
26                 cadena = data.decode("utf-8")
27                 longitud = len(cadena)
28                 conn.sendall(str(longitud).encode("utf-8"))
29                 guardar_en_txt(addr, cadena, longitud)
30                 print(f"Recibido: '{cadena}' -> Respondido: {longitud}")
31     except KeyboardInterrupt:
32         print("\nServidor detenido")
```

En el código podemos ver cómo recibe cualquier conexión y expone el puerto 5050, dentro de la definición `guardar_txt` se guardaría la información de la hora actual, la IP de la conexión, el puerto utilizado y la línea de caracteres con su longitud. Una vez ejecutado el servidor mostraría en la terminal en qué puerto y IP se ejecuta el servidor esperando una solicitud.



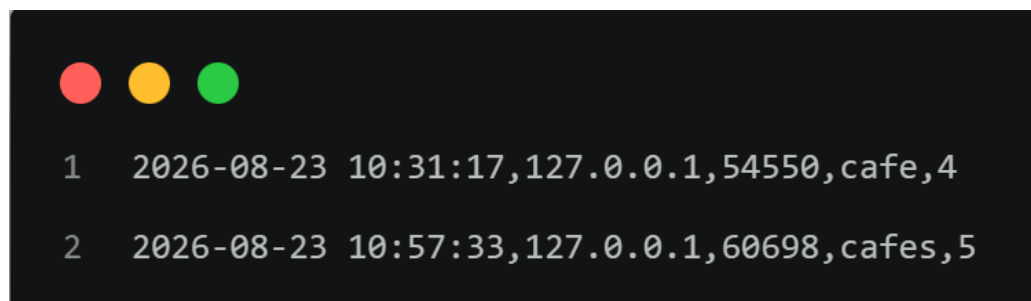
```
PS C:\Users\justi\OneDrive\Documents\escritorio 22_08_26\P002> & C:\Users\justi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/justi/OneDrive/Documents/escritorio 22_08_26/P002/servidor_tcp_mono.py"
Servidor escuchando en 0.0.0.0:5050
Conexión establecida desde ('127.0.0.1', 60698)
Recibido: 'cafes' -> Respondido: 5
```

Después de ejecutar el servidor también ejecutamos el cliente y mandamos la solicitud y nos mostraría el resultado como en la imagen anterior.



```
1 import socket
2
3 HOST = "127.0.0.1"
4 PORT = 5050
5
6 with socket.create_connection((HOST, PORT)) as s:
7     mensaje = input("Ingrese una cadena: ")
8     s.sendall(mensaje.encode("utf-8"))
9     data = s.recv(1024)
10    print(f"Tamaño recibido del servidor: {data.decode('utf-8')}")
11
```

Una vez terminado el proceso de la solicitud la información guardada en el `archivo.txt` se guarda así:

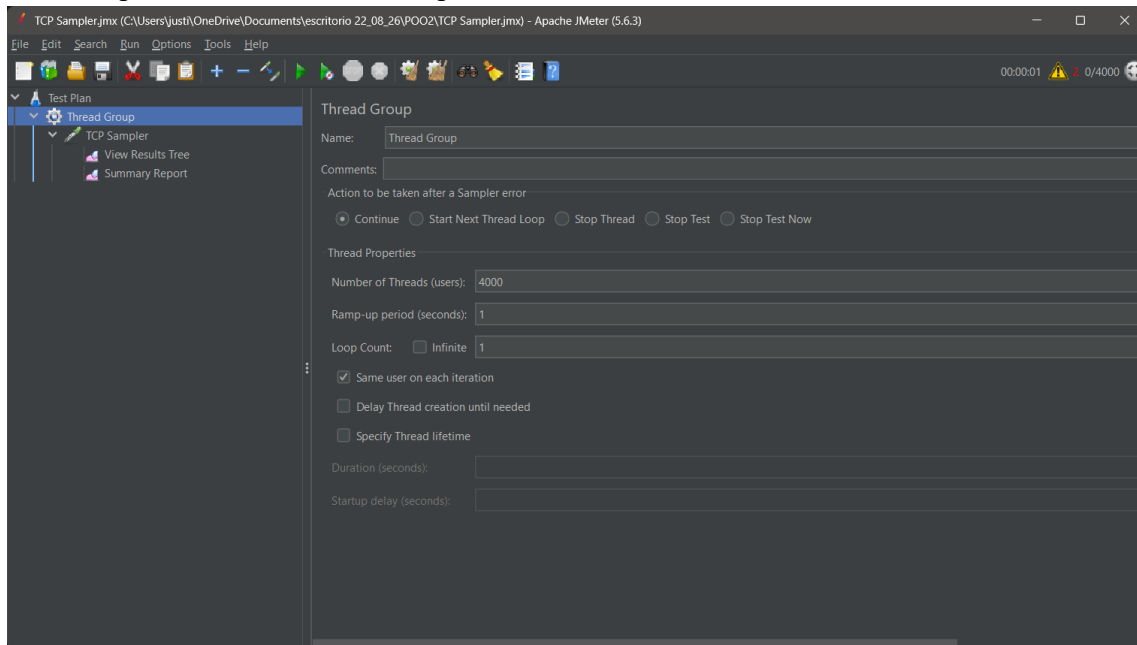


```
1 2026-08-23 10:31:17,127.0.0.1,54550,cafe,4
2 2026-08-23 10:57:33,127.0.0.1,60698,cafes,5
```

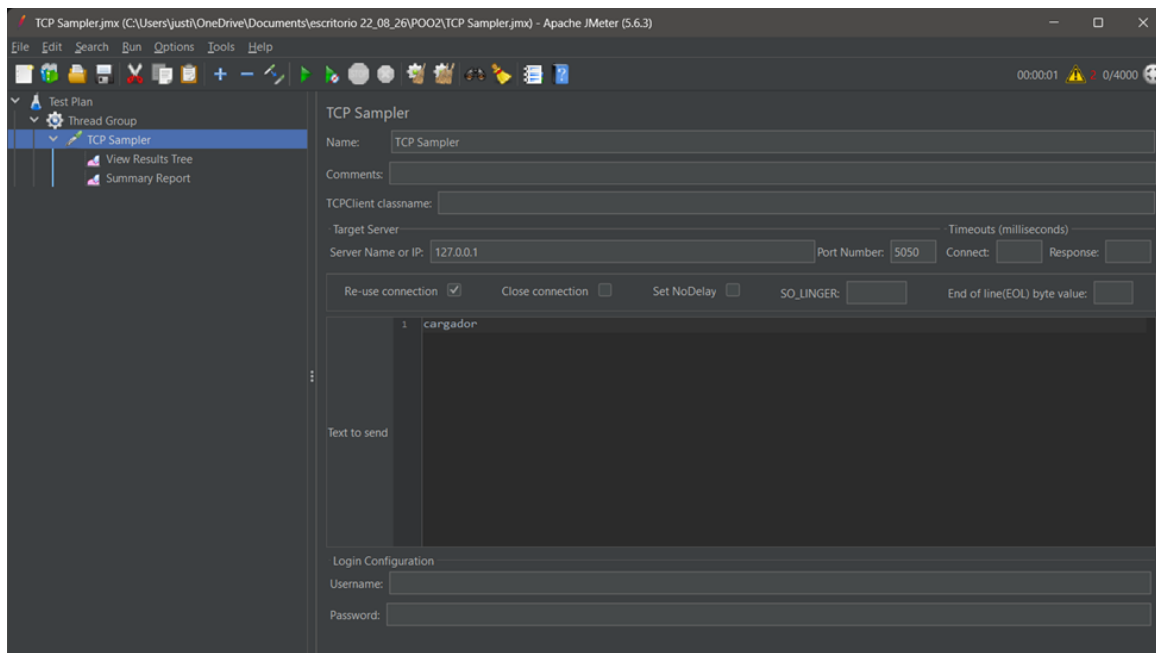
Ahora en el servidor multihilos es el siguiente y debe realizar lo mismo que el servidor mono pero aquí vamos a realizar varias peticiones con la aplicación de Jmeter.

En Jmeter vamos a crear un nuevo test plan y tenemos que configurarlo de la siguiente manera:

- Creamos un multi Thread Group y en el número de Threads en este caso ponemos 4000 que serán las solicitudes que se van a realizar al servidor.

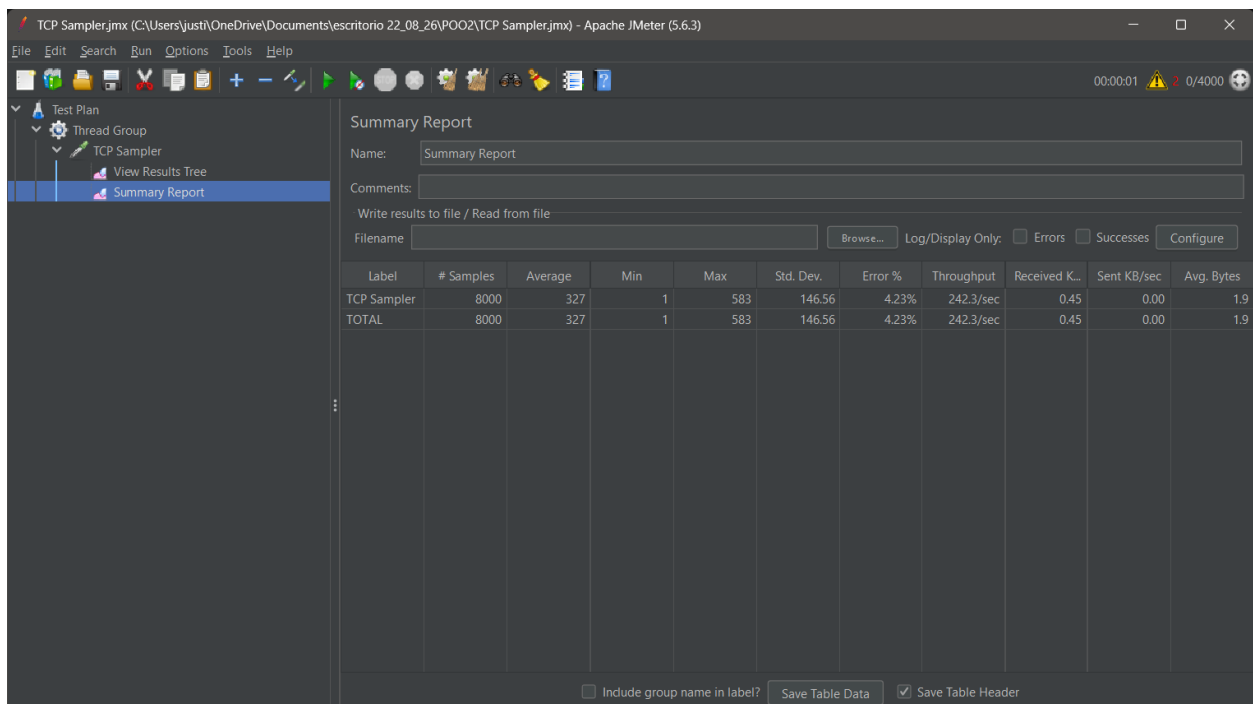
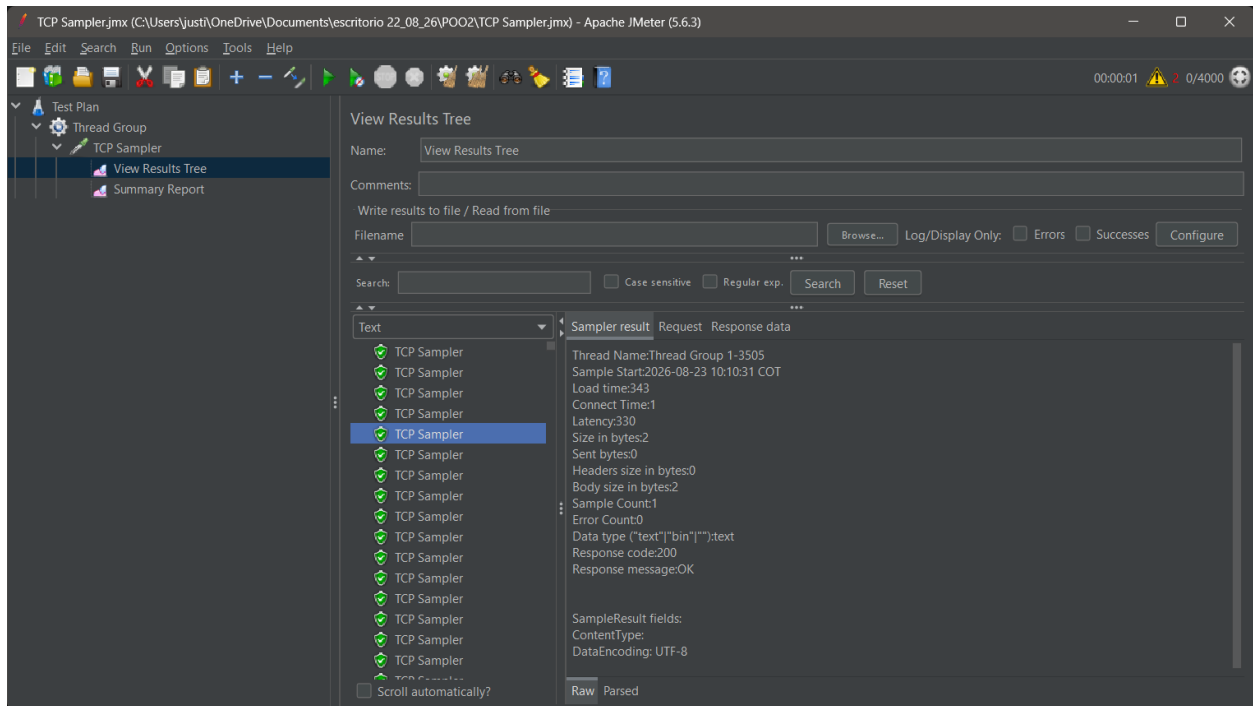


- Ahora buscamos un tcp para hacer la prueba y insertamos los siguientes datos en server Name o IP ponemos nuestra IP local que vamos a realizar la prueba y también el Port



Number que es el 5050 ya en la parte del Text to Send ponemos la palabra que deseamos en este caso fue cargador.

- Ahora añadimos dos reportes para poder ver los resultados uno que es el View results tree y el Summary report que nos permite ver que errores sucedieron o también ver las medidas estadísticas de error o medias, varianzas etc.



Con el servidor multihilos se va a implementar una cola para poder atender todas las peticiones y guardar la información necesaria.



```
1  import socket
2  import threading
3  import queue
4  import re
5  import logging
6  from concurrent.futures import ThreadPoolExecutor
7  from datetime import datetime
8
9  HOST = '0.0.0.0'
10 PORT = 5050
11 MAX_WORKERS = 50          # tamaño del pool de threads
12 PERSIST_FILE = 'resultados.txt'
13 QUEUE_MAXSIZE = 2000      # tamaño máximo de la cola de espera
14
15 logging.basicConfig(
16     level=logging.INFO,
17     format='%(asctime)s [%(threadName)s] %(message)s'
18 )
19 logger = logging.getLogger(__name__)
20
21 # Cola de mensajes en espera para persistencia
22 persist_queue = queue.Queue(maxsize=QUEUE_MAXSIZE)
23
24 # Solo letras del alfabeto (mayúsculas o minúsculas)
25 ALPHA_PATTERN = re.compile(r'^[A-Za-z]+$')
```

Definimos los **MAX_WORKERS** el cual es la cantidad de hilos que van a trabajar las peticiones que llegarían y **QUEUE_MAXSIZE** es el tamaño máximo de la cola de espera que tendrá el servidor para esperar las peticiones.

Con **login.basicConfig** nos mostrara el formato de salida y cuál es el puerto de escucha del servidor cuando esté funcionando. Ahora realizamos la persistencia de los mensajes con **persist_queue** y tiene como valor el tamaño máximo de la cola de espera. Finalmente con **ALPHA_PATTERN** tenemos todas las letras para su lectura y guardado de datos.

```
1 def count_last_char(text: str) -> int:
2     """Cuenta cuántas veces aparece el último carácter en la cadena."""
3     last_char = text[-1]
4     return text.count(last_char)
```

Con la definición de **count_last_char** nos ayuda a ver cuantas veces aparece el ultimo carácter en la palabra seleccionada y retorna su cantidad.

```
1 def persist_worker():
2     """
3     Hilo único dedicado que consume la cola y escribe en el archivo.
4     Al ser el único consumidor, se evitan condiciones de carrera en el
5     archivo sin necesidad de locks explícitos sobre el I/O.
6     """
7     with open(PERSIST_FILE, 'a', encoding='utf-8') as f:
8         while True:
9             item = persist_queue.get()
10            if item is None: # señal de apagado
11                persist_queue.task_done()
12                break
13            word, count, ts = item
14            f.write(f"{ts}\t{word}\t{count}\n")
15            f.flush()
16            persist_queue.task_done()
```

La definición **PERSISTEN_WORLER** se encarga de gestionar la cola de peticiones del servidor y forzar su escritura en el disco con **flush** además de no encontrarse nada en la cola no se ejecuta y termina su tarea.

Con `handle_client` es la defición principal aquí manejamos la conexión con una espera de 10 segundos para evitar conexiones caídas o colgadas, se revisa si pertenece todo el texto a las letras del alfabeto sino corresponde no acepta la conexión, solo lee hasta 4096 bytes por mensaje.

Con el texto de la petición lo decodifica de bytes a un string además de asegurarse de no romperse si existen bytes inválidos y espacios en los extremos. Envía la respuesta antes de guardar la información para ganar velocidad de respuesta para el cliente.

Si la cola se llena se descarta el registro, sino es el caso la envía al otro hilo de dicado a guardar la información en el txt y al final se cierra la conexión con el cliente.

```
1  def handle_client(conn: socket.socket, addr):
2      """Maneja una conexión de cliente (ejecutado por un hilo del pool)."""
3      try:
4          conn.settimeout(10)
5          data = conn.recv(4096)
6          if not data:
7              return
8
9          text = data.decode('utf-8', errors='replace').strip()
10
11         if not text or not ALPHA_PATTERN.match(text):
12             response = "ERROR: el mensaje debe contener solo letras del alfabeto\n"
13             conn.sendall(response.encode('utf-8'))
14             logger.warning(f"Mensaje invalido de {addr}: {text!r}")
15             return
16
17         count = count_last_char(text)
18         response = f"{count}\n"
19         conn.sendall(response.encode('utf-8'))
20
21         # Encolar para persistencia (no bloquea la respuesta al cliente)
22         ts = datetime.now().isoformat()
23         try:
24             persist_queue.put_nowait((text, count, ts))
25         except queue.Full:
26             logger.error("Cola de persistencia llena, se descarta el registro")
27
28         logger.info(
29             f"{addr} -> {text!r} : ultimo caracter '{text[-1]}' aparece {count} veces"
30         )
31
32     except socket.timeout:
33         logger.warning(f"Timeout con {addr}")
34     except Exception as e:
35         logger.exception(f"Error manejando a {addr}: {e}")
36     finally:
37         conn.close()
```

Finalmente llegamos al main y su comportamiento es el siguiente primero se inicia el hilo **PERSIST_WORKER** el cual se encarga de escribir todos los datos en el disco que llegan desde la cola. Luego se crea un socket del servidor utilizando un TCP/IPv4 configurado para reutilizar le puerto si el programa se reinicia rapidamente.

El servidor se encuentra en un bucle infinito de escucha para las conexiones cada vez que llega un cliente no se procesa principalmente con el hilo principal sino se delega a un **THREADPOOLEXECUTOR** lo cual permite manejar multiples clientes en paralelos sin crear hilos ilimitados controlando mejor los recursos del sistema.

```
1  def main():
2      # Arrancar el hilo escritor (consumidor de la cola)
3      writer_thread = threading.Thread(
4          target=persist_worker, name="PersistWriter", daemon=True
5      )
6      writer_thread.start()
7
8      server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9      server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
10     server_sock.bind((HOST, PORT))
11     server_sock.listen(256)
12     logger.info(f"Servidor escuchando en {HOST}:{PORT}")
13
14     with ThreadPoolExecutor(max_workers=MAX_WORKERS, thread_name_prefix="Worker") as pool:
15         try:
16             while True:
17                 conn, addr = server_sock.accept()
18                 pool.submit(handle_client, conn, addr)
19         except KeyboardInterrupt:
20             logger.info("Apagando servidor...")
21         finally:
22             persist_queue.put(None) # señal de apagado para el escritor
23             writer_thread.join(timeout=5)
24             server_sock.close()
25
26
27 if __name__ == '__main__':
28     main()
```


Aquí vemos el registro de todas las operaciones del servidor y su ejecución en una terminal.

The image shows a Windows 10 desktop with two Visual Studio Code windows open. The top window is titled "POO2" and shows the source code for a Python TCP server named "server_tcp_multi.py". The code includes imports for socket, threading, queue, re, logging, concurrent.futures, and datetime. It defines a "logging.basicConfig" function, a "persist_queue" variable, and a "def count_last_char" function. The main logic is in the "def persist_worker" function, which listens for connections and processes data. The terminal output shows the server listening on 0.0.0.0:5050.

The bottom window is also titled "POO2" and shows the source code for a Python TCP client named "cliente_tcp.py". The code includes imports for socket, threading, queue, re, logging, concurrent.futures, and datetime. It defines a "logging.basicConfig" function, a "persist_queue" variable, and a "def count_last_char" function. The main logic is in the "def persist_worker" function, which sends data to the server and receives a response. The terminal output shows the client sending data to the server, with the server responding with "cargador" and a count of 2.